1-Number of Zeros in a Given Array

Started on	Wednesday, 19 August 2026, 8:51 AM
State	Finished
Completed on	Wednesday, 19 August 2026, 8:56 AM
Time taken	5 mins 19 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct | Mark 1.00 out of 1.00 | Flag question

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

```c
#include<stdio.h>

int func(int arr[], int first, int last){
    if(first > last)
        return 0;

    if(arr[last] != 0)
        return 0;

    if(arr[first] == 0)
        return (last-first+1);

    int mid = first + (last-first) / 2;

    return func(arr, first, mid) + func(arr, mid+1, last);
}

int main(){
    int m;
    scanf("%d", &m);
    int arr[m];
    for(int i=0; i<m; i++){
        scanf("%d", &arr[i]);
    }
    printf("%d\n", func(arr, 0, m-1));
    return 0;
}
```

	Input	Expected	Got	
✔	5 1 1 1 0 0	2	2	✔
✔	10 1 1 1 1 1 1 1 1 1 1	0	0	✔
✔	8 0 0 0 0 0 0 0 0 0	8	8	✔
✔	17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0	2	2	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

2-Majority Element

Started on	Wednesday, 26 August 2026, 1:37 PM
State	Finished
Completed on	Wednesday, 26 August 2026, 1:39 PM
Time taken	2 mins 8 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

```
Input: nums = [3,2,3]
Output: 3
```

Example 2:

```
Input: nums = [2,2,1,1,1,2,2]
Output: 2
```

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int majorityElement(int nums[], int n){
    int candidate = nums[0];
    int count = 1;

    for (int i = 1; i < n; i++){
        if (nums[i] == candidate)
            count++;
        else{
            count--;
            if (count == 0){
                candidate = nums[i];
                count = 1;
            }
        }
    }
    return candidate;
}

int main(){
    int n;
    scanf("%d", &n);
    int nums[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &nums[i]);
    printf("%d\n", majorityElement(nums, n));
    return 0;
}
```

	Input	Expected	Got	
✔	3 3 2 3	3	3	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

3-Finding Floor Value

Started on	Wednesday, 19 August 2026, 9:24 AM
State	Finished
Completed on	Wednesday, 19 August 2026, 9:32 AM
Time taken	8 mins 4 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```c
#include<stdio.h>

int func(int arr[], int n, int x){
    int first = 0, last = n-1;
    int floorval = -1;

    while( first <= last){
        int mid = first + (last-first) / 2;

        if(arr[mid] == x)
            return arr[mid];

        else if(arr[mid] < x){
            floorval = arr[mid];
            first = mid+1;
        }

        else
            last = mid-1;
    }

    return floorval;
}

int main(){
    int n, x;
    scanf("%d", &n);
    int arr[n];
    for(int i=0; i<n; i++){
        scanf("%d", &arr[i]);
    }
    scanf("%d", &x);

    printf("%d\n", func(arr, n, x));
    return 0;
}
```

	Input	Expected	Got	
✔	6 1 2 8 10 12 19 5	2	2	✔
✔	5 10 22 85 108 129 100	85	85	✔
✔	7 3 5 7 9 11 13 15 10	9	9	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

4-Two Elements sum to x

Started on	Wednesday, 19 August 2026, 9:23 AM
State	Finished
Completed on	Wednesday, 26 August 2026, 1:33 PM
Time taken	7 days 4 hours
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
int element1, element2;
int binarySearch(int arr[], int low, int high, int key){
    if (low > high)
        return -1;
    int mid = (low + high) / 2;
    if (arr[mid] == key)
        return mid;
    if (arr[mid] > key)
        return binarySearch(arr, low, mid - 1, key);
    return binarySearch(arr, mid + 1, high, key);
}

int findPair(int arr[], int n, int x, int i){
    if (i >= n - 1)
        return 0;
    int required = x - arr[i];
    int pos = binarySearch(arr, i + 1, n - 1, required);
    if (pos != -1){
        element1 = arr[i];
        element2 = arr[pos];
        return 1;
    }
    return findPair(arr, n, x, i + 1);
}

int main(){
    int n, x;
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    scanf("%d", &x);
    if (findPair(arr, n, x, 0)){
        printf("%d\n", element1);
        printf("%d\n", element2);
    }
    else
        printf("No\n");
    return 0;
}
```

	Input	Expected	Got	
✔	4 2 4 8 10 14	4 10	4 10	✔
✔	5 2 4 6 8 10 100	No	No	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

5-Implementation of Quick Sort

Started on	Wednesday, 26 August 2026, 1:34 PM
State	Finished
Completed on	Wednesday, 26 August 2026, 1:36 PM
Time taken	1 min 40 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Write a Program to Implement the Quick Sort Algorithm

Input Format:
The first line contains the no of elements in the list-n
The next n lines contain the elements.

Output:
Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

Answer:

```c
#include <stdio.h>

void quickSort(int arr[], int low, int high){
    if (low < high){
        int i = low;
        int j = high;
        int pivot = arr[low];

        while (i < j){
            while (arr[i] <= pivot && i < high)
                i++;

            while (arr[j] > pivot)
                j--;

            if (i < j){
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }

        int temp = arr[low];
        arr[low] = arr[j];
        arr[j] = temp;

        quickSort(arr, low, j - 1);
        quickSort(arr, j + 1, high);
    }
}

int main(){
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    quickSort(arr, 0, n - 1);
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

	Input	Expected	Got	
✔	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✔
✔	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✔
✔	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.